

— IoT Devices

Hoofdstuk 5: SPI

L. Espeel

hogeschool
VIVES



(bron: grotendeels van R. Buysschaert)

—

SPI
algemene info

SPI (1)

- SPI = Serial Peripheral Interface
- Een communicatiebus die data **serieel** verstuurt, is net als I²C een **synchrone** bus. Ze heeft dus een kloksignaal. Het ritme van de klok bepaalt het ritme van de datatransfert.
- Wordt, naast I²C, typisch gebruikt om te communiceren tussen IC's op éénzelfde print.

3

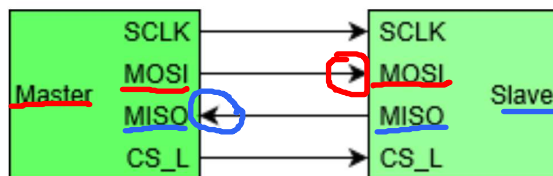
SPI (2)

- Veelgebruikte naamgevingen:
 - MOSI = Master Output Slave Input
 - MISO = Master Input Slave Output
 - CS = Chip Select
 - SCK = Serial Clock
- Alternatieve naamgevingen:
 - SDO = Serial Data Out (= MOSI)
 - SDI = Serial Data In (= MISO)
 - (N)SS = Slave Select (= CS)

4

SPI (3)

Verschillende opstellingen:



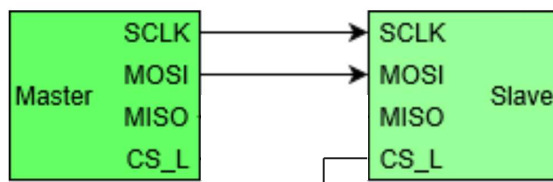
SPI Bus – Simple Point-to-Point
Topology

SPI: Kan maar 1 master hebben, en kan wel meerdere slaafjes hebben

5

SPI (4)

Verschillende opstellingen:



SPI Bus – Simple Point-to-Point
Topology

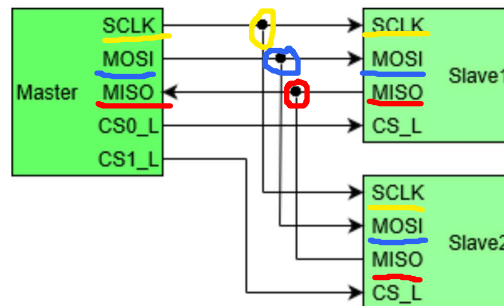
Opmerking: je kan in sommige gevallen ook enkel data van master naar slave sturen en de chip select constant laag houden...

Sommige IC's hebben zelfs geen chip select (APA102C RGB LED).

6

SPI (5)

Verschillende opstellingen:



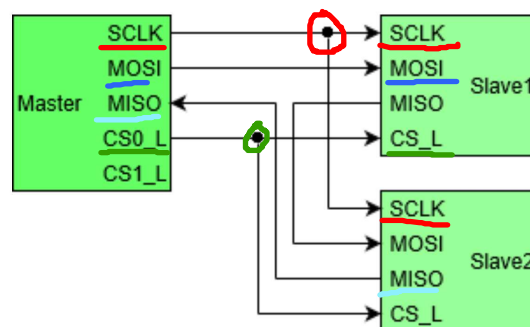
SPI Bus – Star Topology

7

hogeschool
vives

SPI (6)

Verschillende opstellingen:



SPI Bus – Daisy Chain Topology

= kan enkel met apparaten die het ondersteunen

8

hogeschool
vives

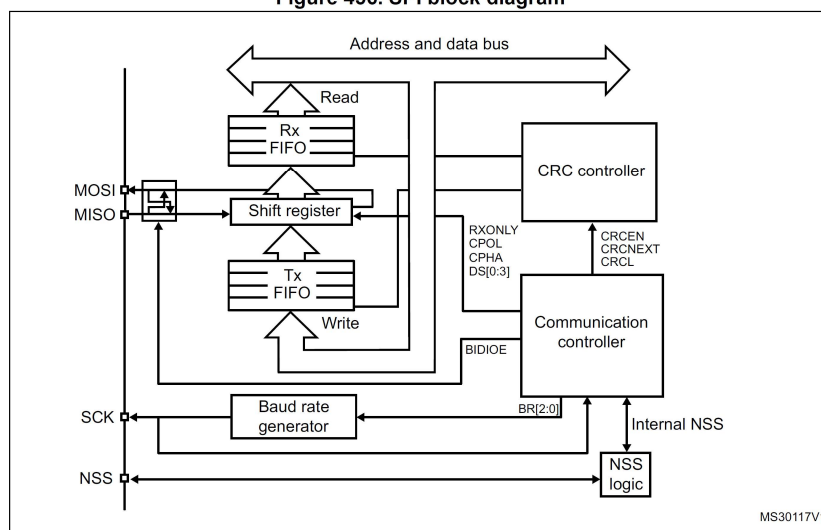
SPI

in de STM32L476RG

9

SPI in de STM32L476RG ⁽¹⁾

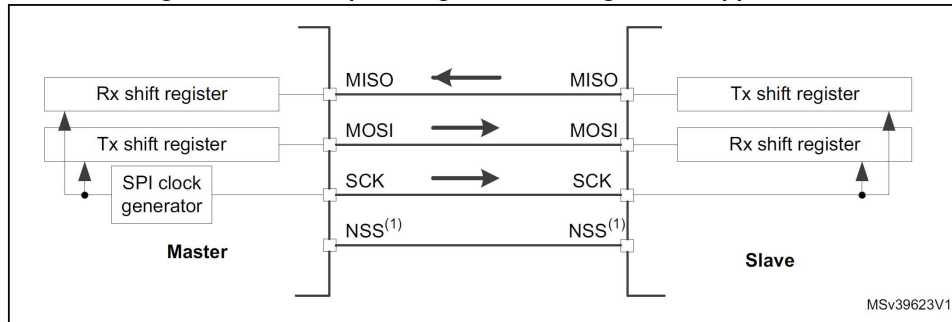
Figure 456. SPI block diagram



10

SPI in de STM32L476RG ⁽²⁾

Figure 457. Full-duplex single master/ single slave application



1. The NSS pins can be used to provide a hardware control flow between master and slave. Optionally, the pins can be left unused by the peripheral. Then the flow has to be handled internally for both master and slave. For more details see [Section 42.4.5: Slave select \(NSS\) pin management](#).

Bron: RM0351 STM32 microelectronics.

11

hogeschool
vives

SPI in de STM32L476RG ⁽³⁾

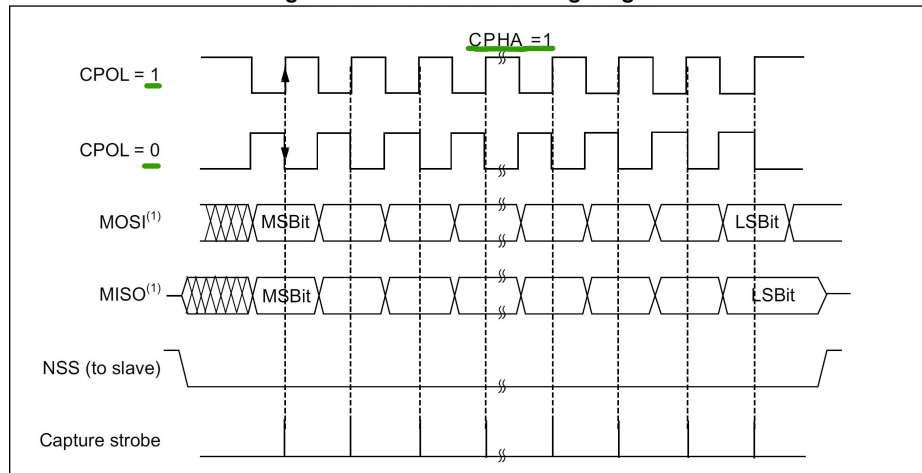
- SPI kan in **4 verschillende modes** communiceren. Die mode wordt ingesteld met de **clock polarity (CPOL)** en **clock phase (CPHA)**.
- Als CPOL nul is, dan is het kloksignaal laag indien er geen data verstuurd wordt. Als CPOL één is, dan is het kloksignaal hoog indien er geen data verstuurd wordt.
- Als CPHA nul is, wordt de eerste flank (stijgend of dalend) van de klok gebruikt om data in te klokken. Als CPHA één is, wordt de tweede flank van de klok gebruikt om data in te klokken.
- Zie afbeelding volgende slides.

12

hogeschool
vives

SPI in de STM32L476RG ⁽⁴⁾

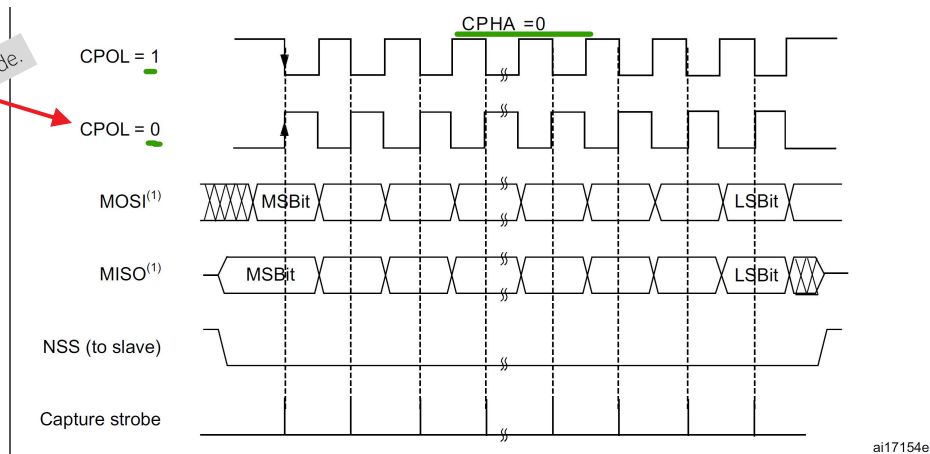
Figure 463. Data clock timing diagram



Bron: RM0351 STMicronics.

SPI in de STM32L476RG ⁽⁵⁾

Veelgebruikte mode.



Bron: RM0351 STMicronics.

SPI in de STM32L476RG ⁽⁶⁾

- CPOL en CPHA kan je instellen in het SPIx->CR1 register.

42.6.1 SPI control register 1 (SPIx_CR1)

Address offset: 0x00

Reset value: 0x0000

Waarvoor zou BR[2:0] dienen?

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDIOE	CRC EN	CRCN EXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR[2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 1 **CPOL**: Clock polarity

0: CK to 0 when idle

1: CK to 1 when idle

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode except the case when CRC is applied at TI mode.

Bit 0 **CPHA**: Clock phase

0: The first clock transition is the first data capture edge

1: The second clock transition is the first data capture edge

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode except the case when CRC is applied at TI mode.

Bron: RM0351 STM32Microelectronics.

15

hogeschool
vives

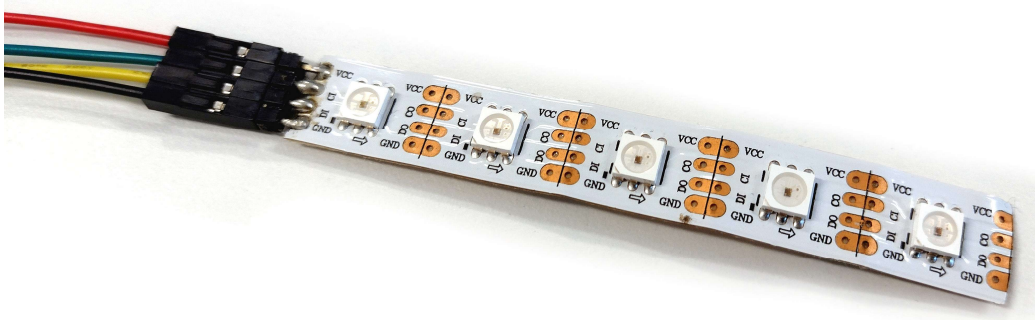
APA102C

16

hogeschool
vives

— APA102C ⁽¹⁾

- De APA102C is een RGB LED die aanstuurbaar is via SPI.
- Je kan er één aparte LED mee aansturen of een volledige LED-strip.
- Werkt met een 'start frame' en een 'end frame' (zie datasheet).

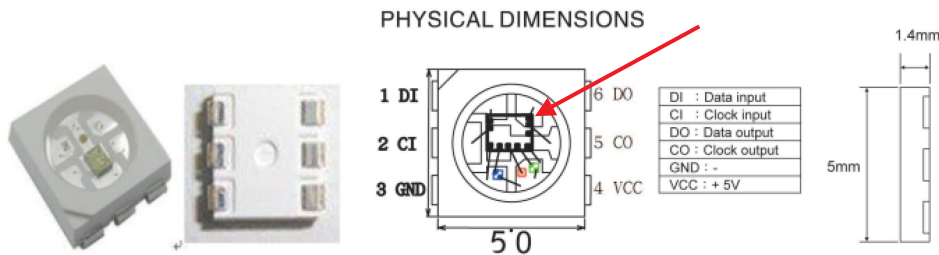


17

hogeschool
vives

— APA102C ⁽²⁾

- In ieder LED zit een chip die de SPI-communicatie en LED-sturing op zich neemt.
- De uitgang van de ene LED kan gekoppeld worden met de ingang van de andere LED.



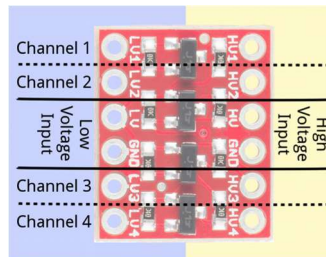
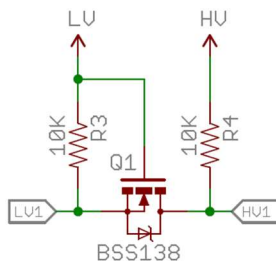
Bron: RM0351 stjj-led.com.

18

hogeschool
vives

Nucleo met APA102C

- Probleem: de LED-strip is 5V gevoed, ook de SPI-lijnen moeten best 5V niveaus behandelen ⇔ microcontroller waar alles op 3V3 werkt.
- Oplossing: logic level converter tussen microcontroller en LED-strip. Werking?



Hoe verbinden?

Microcontroller -> logic level converter:

* MOSI met LV1

* CLK met LV2

* 3V3 met LV

* GND's

Logic level converter -> LED-strip:

* HV1 met DI

* HV2 met CI

* HV met 5V

* GND's

Opmerking:

nog extra aftakking nodig naar 5V voeding!

19

APA102C aansturen via polling

20

— APA102C aansturen via polling ⁽¹⁾

- De eenvoudigste manier om de LED's aan te sturen is via blokkerende functies, via polling dus.

HAL_SPI_Transmit

Function name

HAL_StatusTypeDef HAL_SPI_Transmit (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmit an amount of data in blocking mode.

Parameters

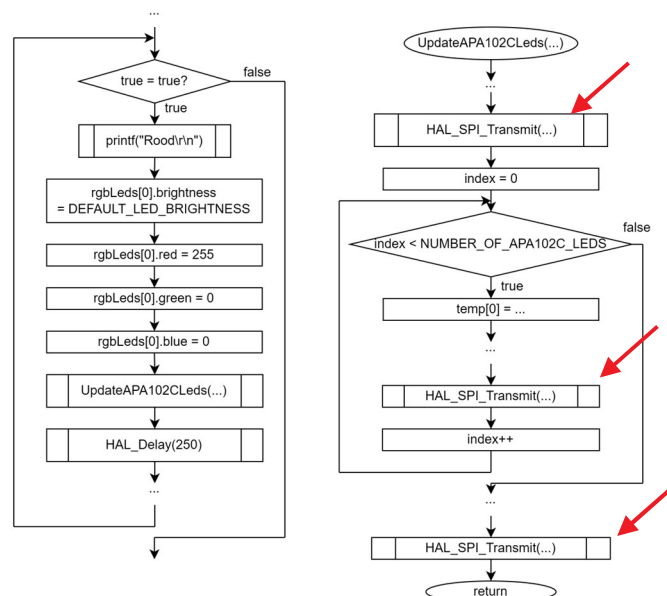
- hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- pData**: pointer to data buffer
- Size**: amount of data to be sent
- Timeout**: Timeout duration

Return values

- HAL**: status

21

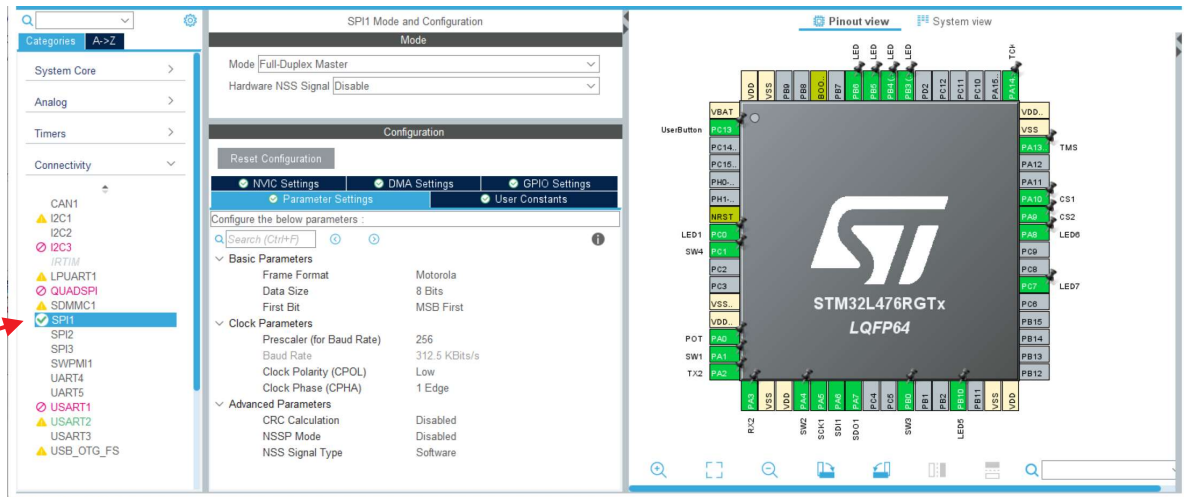
— APA102C aansturen via polling ⁽²⁾



22

— APA102C aansturen via polling (3)

- Stel de SPI-module in via STM32CubeMX:



23

— APA102C aansturen via polling (4)

```
1 #include "stm32l4xx_hal.h"
2 #include "apa102c.h"
3
4 // Verstuur de data over SPI1. Let op: zorg dat SPI1 reeds geïnitieerd is.
5 void UpdateAPA102CLeds(SPI_HandleTypeDef* hspi, APA102C led[])
6 {
7     uint8_t index = 0;
8     uint8_t temp[] = {0, 0, 0, 0};
9
10    HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);    // Start frame.
11
12    for(index = 0; index < NUMBER_OF_APA102C_LEDS; index++)
13    {
14        temp[0] = led[index].brightness | 0xE0;    // 5-bit intensiteit.
15        temp[1] = led[index].blue;    // Blauw.
16        temp[2] = led[index].green;    // Groen.
17        temp[3] = led[index].red;    // Rood.
18
19    }
20    HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);
21
22    temp[0] = 255;
23    temp[1] = 255;
24    temp[2] = 255;
25    temp[3] = 255;
26    HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);    // End frame.
27 }
```

24

APA102C aansturen via DMA

25

— APA102C aansturen via DMA ⁽¹⁾

- De beste manier om de LED's aan te sturen is via DMA-functies, niet-blokkerende functies dus.

HAL_SPI_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with DMA.

Parameters

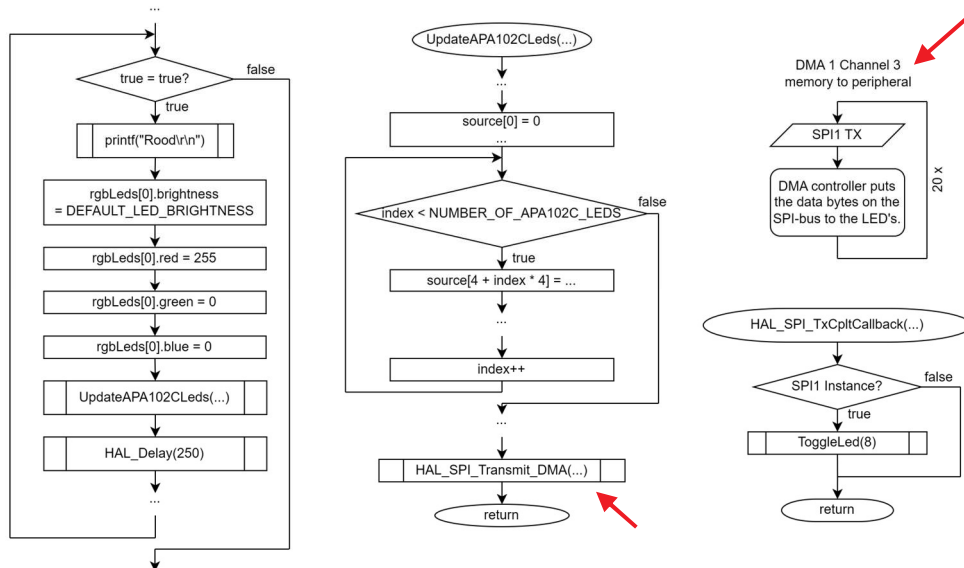
- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

26

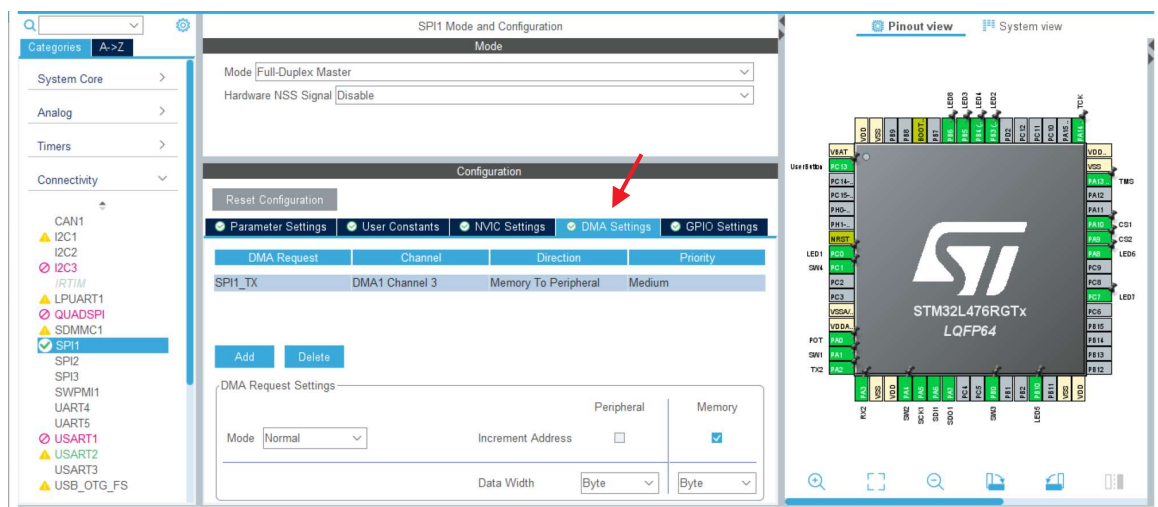
APA102C aansturen via DMA (2)



27

APA102C aansturen via DMA (3)

- Stel de SPI-module in via STM32CubeMX:



28

APA102C aansturen via DMA (4)

```
1 #include "stm3214xx_hal.h"
2 #include "apa102c.h"
3
4 // Verstuur de data over SPI1. Let op: zorg dat SPI1 reeds geïntialiseerd is.
5 void UpdateAPA102CLeds(SPI_HandleTypeDef* hspi, APA102C led[])
6 {
7     uint8_t index = 0;
8     extern uint8_t source[4 + NUMBER_OF_APA102C_LEDS * 4 + 4];
9
10    source[0] = 0; // Start frame.
11    source[1] = 0;
12    source[2] = 0;
13    source[3] = 0;
14
15    for(index = 0; index < NUMBER_OF_APA102C_LEDS; index++)
16    {
17        source[4 + index * 4] = led[index].brightness | 0xE0; // 5-bit intensiteit.
18        source[5 + index * 4] = led[index].blue; // Blauw.
19        source[6 + index * 4] = led[index].green; // Groen.
20        source[7 + index * 4] = led[index].red; // Rood.
21    }
22
23    source[4 + NUMBER_OF_APA102C_LEDS * 4] = 255; // End frame.
24    source[5 + NUMBER_OF_APA102C_LEDS * 4] = 255;
25    source[6 + NUMBER_OF_APA102C_LEDS * 4] = 255;
26    source[7 + NUMBER_OF_APA102C_LEDS * 4] = 255;
27
28    // Alle data verzenden via DMA.
29    HAL_SPI_Transmit_DMA(hspi, source, (4 + NUMBER_OF_APA102C_LEDS * 4 + 4));
30 }
```

29

APA102C aansturen via DMA (5)

```
597 /* USER CODE BEGIN 4 */
598 void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
599 {
600     if(hspi->Instance == SPI1)
601     {
602         ToggleLed(8); // Teken van leven geven...
603     }
604 }
605 /* USER CODE END 4 */
```

30

— APA102C aansturen via DMA ⁽⁶⁾

- Hieronder het scoopbeeld voor het aansturen van 5 APA102C LED's.
- Kan je het 'protocol' erin terugvinden?

